

Plan detallado para desarrollar una plataforma de compañía conversacional

Guía técnica y de producto para construir un MVP local-first preparado para migrar a AWS sin rehacer la arquitectura.

Documento de trabajo

Enfoque: desarrollo local con Docker, backend en FastAPI, frontend en Next.js, patrón de adapters/providers y despliegue progresivo en AWS.

Objetivo: que el documento sirva como guía ejecutable para empezar a programar el proyecto con criterio de MVP, validación temprana y escalado posterior.

Fase 1 Local-first Construcción del MVP	Fase 2 Piloto controlado Despliegue mínimo en AWS	Fase 3 Escalado Matching + IA + operación
---	--	---

1. Resumen ejecutivo

El proyecto debe arrancar como un marketplace simple de conversación humana bajo demanda, no como una app compleja, una red social ni un producto clínico.

La arquitectura recomendada es local-first: se desarrolla y prueba casi todo en local con Docker, PostgreSQL, FastAPI y Next.js; AWS se introduce después mediante adapters y configuración por entorno.

La regla de diseño principal es separar negocio de infraestructura. La aplicación no debe depender directamente de S3, Cognito, SES o Chime, sino de interfaces que luego tengan implementaciones locales y de AWS.

El primer objetivo no es escalar, sino validar: conseguir perfiles reales, reservas reales y aprender qué tipo de conversación se vende mejor.

2. Principios del proyecto

- Empezar pequeño: solo las funciones necesarias para reservar, pagar, conversar y valorar.
- Construir sobre PostgreSQL desde el día 1 para que el paso a RDS sea natural.
- Usar Docker Compose para que desarrollo, pruebas y futura producción compartan una base operativa consistente.
- Trabajar con una API estable y versionada desde el inicio.
- Proteger el alcance del MVP: no introducir IA, app móvil ni matching avanzado antes de validar mercado.

3. Alcance del MVP

El MVP debe incluir únicamente las capacidades mínimas que permitan validar el negocio:

- registro e inicio de sesión
- perfiles de usuarios y companions
- disponibilidad y agenda
- reserva de sesiones
- pagos en modo test
- enlace o sala para videollamada
- reseñas posteriores a la sesión
- panel de administración básico

Qué queda fuera del MVP

- matching con IA
- recomendaciones avanzadas
- app móvil nativa
- mensajería compleja en tiempo real
- analítica profunda

- claims de salud mental o funcionalidad clínica

4. Arquitectura objetivo

La arquitectura debe diseñarse para funcionar localmente y migrar después a AWS sin cambiar la lógica de negocio. La forma correcta de conseguirlo es separar el sistema por capas.

Capa	Responsabilidad	Ejemplos	No debe hacer
API	Expone endpoints y valida requests/responses	auth, profiles, bookings, reviews	contener reglas de negocio o llamadas directas a AWS
Services	Lógica de negocio del dominio	crear reserva, cancelar sesión, publicar perfil	persistir SQL inline ni hablar con servicios externos
Repositories	Acceso a datos	UserRepository, BookingRepository	decidir reglas del negocio
Providers	Integraciones externas intercambiables	StorageProvider, EmailProvider, VideoProvider	filtrar o transformar lógica de dominio

Diagrama lógico

Frontend (Next.js) -> API REST (FastAPI)

API REST -> Services -> Repositories -> PostgreSQL

API REST -> Providers -> almacenamiento, email, video, pagos, autenticación

Configuración por entorno decide si los providers son locales o AWS

5. Stack tecnológico recomendado

Área	Tecnología base	Razón	Sustitución posterior en AWS
Frontend	Next.js + TypeScript	rápido, moderno, bueno para paneles y landing	AWS Amplify Hosting
Backend	FastAPI + Pydantic + SQLAlchemy	Python encaja contigo y acelera API limpia	AWS App Runner o ECS
BD	PostgreSQL en Docker	misma familia que producción	Amazon RDS PostgreSQL
Migraciones	Alembic	control formal del	mismo flujo sobre

		esquema	RDS
Auth	JWT local desacoplado	permite avanzar sin dependencia fuerte	Amazon Cognito
Archivos	storage local con interfaz	barato y simple en dev	Amazon S3
Email	console/fake provider	sin coste al inicio	Amazon SES
Video	URL dummy o proveedor abstracto	se valida el negocio primero	Amazon Chime SDK u otro proveedor
Pagos	Stripe test mode	simula cobro real sin complejidad extra	Stripe producción
Observabilidad	logs locales estructurados	depuración clara	CloudWatch + alarmas

6. Estructura del repositorio

Una estructura limpia desde el primer commit evita retrabajo, facilita el trabajo con Codex y permite que el paso a AWS sea un cambio de adapters y despliegue, no una reescritura.

```

project/
  frontend/
  backend/
    app/
    api/
    core/
    models/
    schemas/
    services/
    repositories/
    providers/
    auth/
    email/
    storage/
    video/

```

payments/

tests/

infra/

docker/

aws/

docker-compose.yml

.env.example

README.md

7. Diseño de interfaces para no rehacer el sistema

El punto más importante del proyecto es que las integraciones externas estén detrás de contratos claros. Eso permite que en local uses implementaciones fake o mínimas y luego sustituyas por AWS.

Interfaz	Métodos mínimos	Evolución típica
StorageProvider	save(file), get_url(path), delete(path)	LocalStorageProvider -> S3StorageProvider
EmailProvider	send_booking_confirmation(), send_cancellation()	ConsoleEmailProvider -> SESEmailProvider
AuthProvider	create_user(), issue_token(), validate_token()	LocalJWTProvider -> CognitoAdapter
VideoProvider	create_room(), get_join_info()	DummyVideoProvider -> ChimeVideoProvider
PaymentProvider	create_checkout(), capture(), refund()	StripeTestProvider -> StripeLiveProvider

8. Modelo funcional del MVP

Usuarios y roles

- Guest: navega, consulta perfiles y crea cuenta.
- User: reserva sesiones, gestiona su agenda y deja reseñas.
- Companion: define perfil público, disponibilidad y preferencias básicas.
- Admin: revisa perfiles, incidencias, pagos y publicaciones.

Flujo principal de negocio

- el usuario entra a la landing y explora perfiles
- crea cuenta o inicia sesión
- selecciona un companion y un slot disponible
- completa el pago en modo test
- recibe confirmación por email
- realiza la sesión
- deja una valoración

9. Modelo de datos inicial

Las tablas iniciales deben responder al negocio real y no intentar cubrir futuras ideas todavía.

Tabla	Campos clave	Propósito	Notas
users	id, email, password_hash, role, status	identidad base	dejar preparado para provider externo
profiles	user_id, display_name, bio, language, country	perfil visible	texto moderable
companion_profiles	user_id, headline, topics, price_per_session, verified	configuración comercial	separa usuario de oferta
availability_slots	companion_id, start_at, end_at, status	agenda	usar UTC internamente
bookings	user_id, companion_id, slot_id, status, total_amount	núcleo transaccional	vincular a pago
payments	booking_id, provider, provider_ref, amount, currency, status	traza de cobro	Stripe test al inicio

reviews	booking_id, rating, comment	prueba social	solo tras sesión completada
verification_documents	user_id, file_path, type, status	moderación y confianza	evitar almacenar sensibles sin necesidad
admin_flags	entity_type, entity_id, reason, status	incidencias	útil desde el principio

10. API inicial recomendada

Auth

POST /api/v1/auth/register

POST /api/v1/auth/login

GET /api/v1/auth/me

Profiles

GET /api/v1/companions

GET /api/v1/companions/{id}

POST /api/v1/profile

PATCH /api/v1/profile

Availability

GET /api/v1/companions/{id}/slots

POST /api/v1/companion/slots

PATCH /api/v1/companion/slots/{id}

Bookings

POST /api/v1/bookings

GET /api/v1/bookings/{id}

POST /api/v1/bookings/{id}/cancel

Payments

POST /api/v1/payments/checkout

POST /api/v1/payments/webhook

Reviews

POST /api/v1/reviews

GET /api/v1/companions/{id}/reviews

Admin

GET /api/v1/admin/users

GET /api/v1/admin/bookings

POST /api/v1/admin/flags

11. Entornos y configuración

Todo debe estar parametrizado por variables de entorno. Esa es la base de la migración posterior.

APP_ENV=local|staging|prod

DATABASE_URL=postgresql+psycopg://...

AUTH_BACKEND=local_jwt|cognito

STORAGE_BACKEND=local|s3

EMAIL_BACKEND=console|ses

VIDEO_BACKEND=dummy|chime

PAYMENT_BACKEND=stripe_test|stripe_live

FRONTEND_BASE_URL=...

Regla práctica

Si un cambio de entorno obliga a modificar servicios de negocio, la arquitectura está mal. Solo deberían cambiar providers, variables de entorno y despliegue.

12. Roadmap detallado de construcción

Hito	Resultado esperado
Semana 0 - Preparación	definir nombre del proyecto, repositorio, tablero de tareas, .env.example, convenciones de ramas, criterios de Done y plantillas para issues/PRs
Semana 1 - Base técnica	levantar docker-compose con frontend, backend y PostgreSQL; configurar Alembic, healthchecks, linting, formato, tests básicos y

	CI mínima
Semana 2 - Dominio inicial	implementar usuarios, perfiles, companions y autenticación local; construir landing simple y panel de perfil
Semana 3 - Agenda y reservas	slots de disponibilidad, listado de companions, detalle de perfil, creación y cancelación de bookings
Semana 4 - Pagos y notificaciones	integrar Stripe test, emails por consola o sandbox, estados de booking y pantalla de confirmación
Semana 5 - Admin y reseñas	moderación básica, gestión de flags, reseñas, dashboard simple de incidencias
Semana 6 - Estabilización	tests de flujo completo, hardening básico, limpieza técnica, demo grabada y checklist para piloto

13. Flujo de trabajo recomendado con Codex

- Tú defines el objetivo de cada iteración con tickets pequeños y verificables.
- Codex implementa cambios en ramas cortas: una feature, un bug o una mejora cada vez.
- Cada ticket debe tener entrada, salida esperada y criterios de aceptación.
- Todo cambio debe terminar con pruebas locales, revisión manual y commit limpio.
- Evita pedir a Codex 'haz toda la plataforma'; divide por módulos y por incrementos pequeños.

Formato sugerido para los tickets

Título: Implementar endpoint de creación de slots

Contexto: companions necesitan publicar disponibilidad

Entrada: companion autenticado, fecha y rango horario

Salida: slot persistido con estado available

Done: tests unitarios, validación de solapamientos, respuesta 201

14. Estrategia de despliegue progresivo en AWS

AWS debe entrar por capas, cuando el MVP ya exista. La migración recomendada no es 'big bang', sino incremental.

Componente	En local	Primer paso en AWS	Objetivo final
Frontend	Next.js local	Amplify Hosting	Amplify + dominio + CDN
Backend	FastAPI en Docker	App Runner	App Runner o ECS con CI/CD
Base de datos	PostgreSQL Docker	RDS pequeño	RDS con backups y endurecimiento
Archivos	carpeta local	bucket S3 privado	S3 + URLs firmadas
Auth	JWT local	Cognito en staging	Cognito en producción
Email	console/fake	SES sandbox	SES producción
Video	dummy/external link	proveedor real en staging	Chime SDK o equivalente

Orden recomendado de migración

- 1. Desplegar frontend y backend
- 2. mover PostgreSQL a RDS

- 3. mover archivos a S3
- 4. introducir Cognito
- 5. activar SES
- 6. conectar proveedor real de videollamada

15. Seguridad y cumplimiento mínimo

- usar HTTPS en cualquier entorno público
- almacenar contraseñas con hash fuerte
- registrar auditoría básica de acciones críticas
- definir política de moderación y abuso
- pedir consentimiento para tratamiento de datos
- evitar mensajes o claims clínicos; el producto es conversación, no terapia
- revisar RGPD antes de abrir a usuarios reales

16. Métricas que importan después del MVP

- visitante -> registro
- registro -> primera reserva
- primera reserva -> segunda reserva
- ratio de cancelación
- ingreso por usuario
- tiempo medio hasta primera reserva
- valoración media de companions
- número de companions activos por semana

Interpretación

Estas métricas deben responder una pregunta comercial, no solo técnica: si la gente entra, reserva y repite. Si no se valida eso, añadir IA solo aumentará el coste y la complejidad.

17. Riesgos principales y mitigación

Riesgo	Impacto	Mitigación
Alcance excesivo	el MVP se vuelve demasiado grande	proteger backlog y congelar funcionalidades no esenciales
Dependencia prematura de AWS	tiempo invertido en infra en lugar de producto	usar adapters y posponer migración
Marketplace vacío	no hay suficientes companions o usuarios	lanzar con oferta curada y nicho claro
Confusión con terapia o dating	mal posicionamiento del producto	mensajes claros: conversación y compañía, no clínica ni citas

Código difícil de migrar	acoplamiento a servicios concretos	interfaces, variables de entorno y tests
--------------------------	------------------------------------	--

18. Checklist de inicio

- ☐ crear repositorio y estructura base
- ☐ definir .env.example
- ☐ montar docker-compose
- ☐ crear esquema inicial con Alembic
- ☐ implementar health endpoint
- ☐ implementar registro/login local
- ☐ crear entidad users y profiles
- ☐ levantar landing y panel simple
- ☐ documentar comandos de arranque
- ☐ abrir tablero de tickets de la semana 1

19. Conclusión operativa

El camino correcto no es empezar por AWS, ni por IA, ni por escalar. El camino correcto es construir un MVP local-first, con arquitectura limpia, contratos claros y despliegue progresivo. Si se sigue este plan, el proyecto puede arrancar con coste bajo, validarse rápido y migrar a AWS sin rehacer la base del sistema.